Data Handling and Exploratory Data Analysis (Lab 1)

CS 53744 Machine Learning Project

Due Date : 11:59 PM, September 16, 2026

Instructor: Prof. Jongmin Lee

How to use this note:

- Read it before the lab session. The lab will not repeat this material.
- Covered: auditing, type coercion and imputation, outlier detection, and group summaries. The emphasis is on what each method assumes and where it breaks.
- Not covered: code and function calls. See the docstrings in the skeleton `src/lab01.py`.
- Rules for this lab (environment, tasks, submission) are in the `README.md` in this folder. Course-wide policy — grading and AI use — was given in the policy slides of the first class and is not repeated here.

1 Learning objectives

1. Check size, value types, duplicates, and missingness before modifying the raw data. (Task 1)
2. Run the cleaning procedure in the order the spec states, and explain why the order is fixed. (Task 2)
3. Flag outliers with the IQR rule, and state the rule's assumptions and limits. (Task 3)
4. Summarize a table by group and read the result together with its counts. (Task 4)
5. Match a question about one table to a bar chart, a histogram, or a boxplot. (report figures 1–3)

2 Background: why a project starts here

When a machine-learning project starts, the tempting first question is which model to use. In practice, what decides the outcome usually happens earlier than that.

The data you are handed was collected by somebody else, and not for the purpose of analysis. The cafe sales records in this lab are a good example. That table exists to process orders and print receipts. So when a payment fails once and goes through on the second try, the same order is stored twice; the price is saved as the text `"4,500"` because that is how it appeared on screen; and fields the staff did not ask about are left empty. None of this was a problem for the people who produced the records. It becomes a problem only for whoever tries to compute something from the table later.

Use that table as it is, and the model learns how the data broke rather than how the cafe sells. What makes this harder is that it is difficult to notice. The program does not stop, and numbers come out. Whether those numbers are right is something you only find out by checking.

Every project therefore starts in roughly the same way. Look at what the table contains, count what is broken, repair it in a fixed order, and summarize the result to

check the repair. Lab 1 is those four steps. The phrase "in a fixed order" matters more than it sounds. The same data and the same statistics give different answers when the steps are reordered. That is why the cleaning procedure in this lab has its order written into the spec, and why grading looks at the order as well as the result. The same thing happens on a team: without a written procedure, two people build different tables from the same data.

The four steps of Lab 1:

- **Audit:** count what the table contains and what is broken. No value changes. (Task 1)
- **Clean:** apply the five steps of the spec, in order. (Task 2)
- **Outliers:** flag unusually large or small values. Flagging is not deleting. (Task 3)
- **Summarize:** group by category and read count, mean, and sum together. (Task 4)

3 Key concepts

3.1 The audit: count before you modify

Before changing any value, count the state of the raw table (Figure 1).

Three counts:

- **Missing values:** count per column, not one total. A column that fails because a sensor broke and a column that is empty because the field was optional call for different repairs. This is `summarize_missing` in Task 1.
- **Value types:** check that each column's type matches its contents. A numeric column read as text is the most common case.
- **Duplicates:** check for rows identical in every column. Duplicates inflate row counts and distort column statistics.

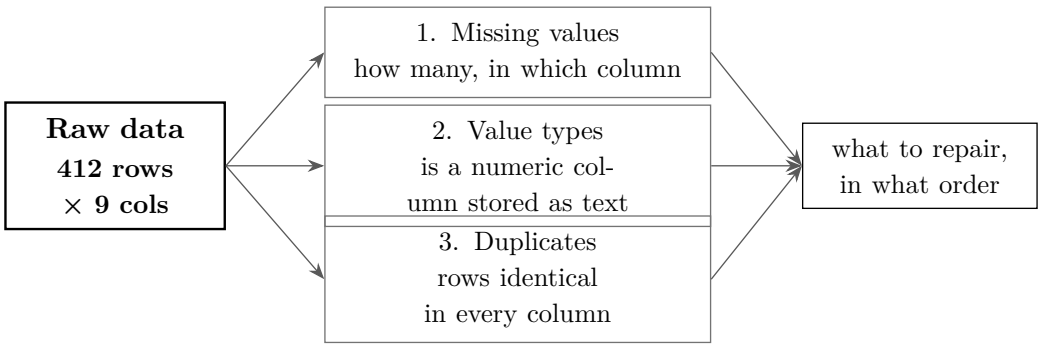

Figure 1. The three checks of an audit. Count missing values, value types, and duplicates on the raw table. The output is a repair plan, not repaired data, and no value changes here.

Watch out:

- An audit changes no values. It produces the evidence you will later use to justify each repair.
- The duplicate check catches only rows identical in every column. The same order recorded twice with different timestamps passes through. Catching that requires knowing how the data was collected.

3.2 Type coercion and imputation

Type coercion:

- Turn strings back into numbers (`"4,500"` → `4500.0`). Finish this before computing anything.
- When a value does not match the conversion rule, raise an error rather than turning it silently into a missing value. One cell reading `4,500 Won` that stops the program is easy to trace; the same cell turned into a missing value and later filled with a column mean is not.

Imputation: two decisions:

1. **Which statistic.** Use the median when values are skewed or contain extremes. The median of $\{1, 2, 100\}$ is 2 and the mean is 34.3 (Figure 2). For a bounded, roughly symmetric quantity such as a 1–5 rating, the mean is fine.
   → This is the basis for the spec assigning the median to `quantity` and the mean to `customer_rating`.
2. **Computed on what.** The deduplicated data, using only values that are present. The statistic goes straight into the result, so when you compute it changes the result.

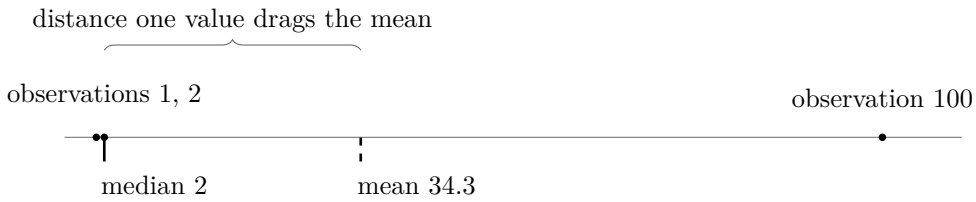

Figure 2. What one extreme value does to the mean. Two of three observations are at most 2, yet the mean climbs to 34.3 while the median stays at 2. This is the basis for choosing what to impute a skewed column with.

Assumptions and exceptions:

- Imputation assumes values are missing more or less at random. If customers who meant to give a low rating skipped the question, filling with the mean raises the average above the truth.
- Derive rather than estimate whenever you can. If `total_price` is missing but `unit_price` and `quantity` are in the same row, multiply them. That is step 4 of the spec.

3.3 The IQR rule for outliers

Let Q_1 and Q_3 be the 25th and 75th percentiles, so the middle half of the data lies between them. Define the *interquartile range* as $\mathrm{IQR} = Q_3 - Q_1$, and flag a value x when either condition holds:

$$x < Q_1 - k \cdot \mathrm{IQR} \qquad \text{or} \qquad x > Q_3 + k \cdot \mathrm{IQR}$$

Figure 3 draws both conditions on a boxplot.

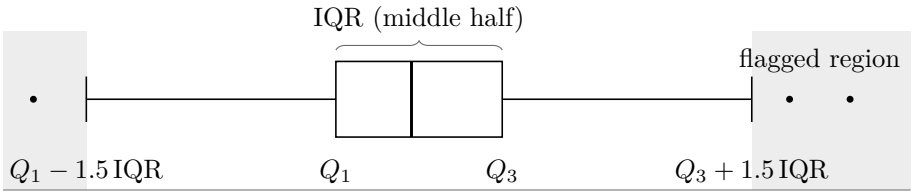

Figure 3. The fences of the IQR rule. The middle half of the data sits inside the box, and the fences stand $1.5 \times$ IQR away from the quartiles. Values outside them are flagged.

What to know:

- **Why $k = 1.5$.** Not derived from theory. It is an empirical setting that flags about 0.7% of normally distributed data. The further the data is from normal, the more that changes.
- **Why quartiles.** Quartiles themselves are not easily moved by a few extreme values.
- **Assumption.** A unimodal, roughly symmetric distribution. On a right-skewed column the natural tail is flagged too. A very high flagged share means the column violates the assumption, not that it holds that many outliers.
- **Flagged $\neq$ error.** The Lab 1 data contains orders of 120, 150, and 200 cups. All three are flagged and all three are plausible bulk orders. The rule only narrows down what a person has to look at. Decide whether to keep or drop them and give your reason in the report.

3.4 Group summaries: split, apply, combine

The work splits into three stages (Figure 4).

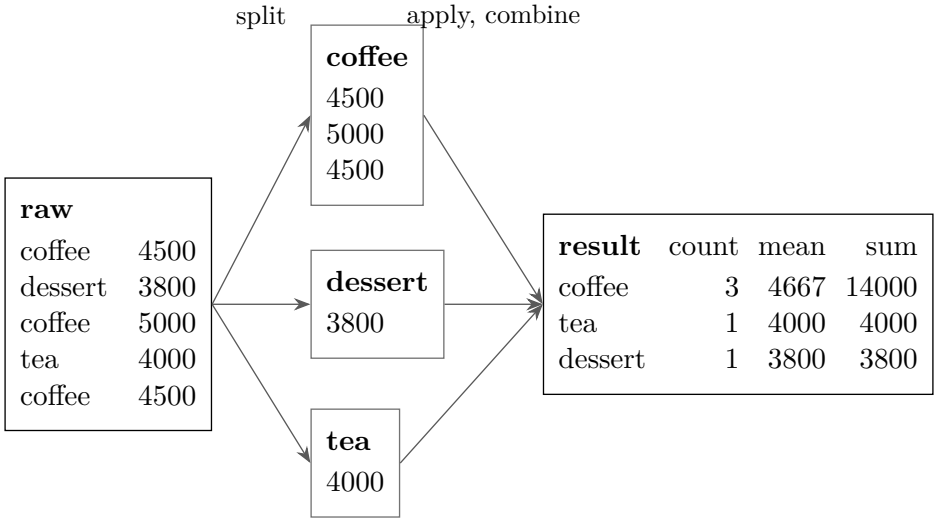

Figure 4. Split, apply, combine. Divide the rows by the grouping key, compute the statistic on each part, and merge the parts into one table. The result is sorted by sum in descending order.

- Sort the result table by sum in descending order (Task 4 spec).
- Read the count column first. A mean over 3 records and a mean over 300 cannot be trusted equally. That is why Task 4 asks for count, mean, and sum together.

3.5 Choosing the plot

Table 1. Questions matched to plots. Which of the bar chart, histogram, and boxplot answers which question. The three figures in your report correspond to these three rows.

Plot	Question it answers	Report figure
Bar chart	How does this one number differ across categories?	No. 1: missing values per column
Histogram	How is this one numeric quantity spread out? (skew, outliers, which summary to use)	No. 2: `total_price` distribution
Boxplot	How do the distributions differ across categories? (median, IQR, flagged points)	No. 3: `customer_rating` by category

- Histogram for the shape of one distribution; boxplot for comparing categories.
- You should be able to write one sentence per figure saying what question it answers, and that sentence belongs in the report. A figure never referred to in the text earns no credit.

4 Worked example: four rows through the procedure

Table 2. The input of the worked example. One duplicated pair, one price stored as text, and one missing quantity. It is the Lab 1 dataset's real problems reduced to four rows.

item	price (text)	qty	note
americano	`"4,500"`	3	
latte	`"5,000"`	1	
latte	`"5,000"`	1	identical to the row above in every field
scone	`"3,800"`	*missing*	

Following the spec order:

1. Drop duplicates: the second latte row goes, leaving three rows.
2. Coerce types: prices become 4500, 5000, 3800.
3. Impute: the median of the observed quantities $\{3, 1\}$ is 2, so the scone gets quantity 2.
4. Derive totals: price $\times$ qty $=$ 13500, 5000, 7600.
5. Impute ratings: skipped; this toy table has no such column.

Grouped by item and sorted by revenue, descending: **americano 13500, scone 7600, latte 5000.**

Skipping step 1:

- The quantities become $\{3, 1, 1\}$, so the median is 1: the scone gets quantity 1 and revenue 3800.
- The duplicated latte row is still there, so latte comes out at 10000.
- Result: **americano 13500, latte 10000, scone 3800.**

Same data, same statistics, only a different step order. Both the filled value and the category ranking changed.

Mind the integer cast:

> **Spec, step 3**
> Fill missing `quantity` with the **median** of the non-missing quantities (computed after step 1), then cast `quantity` to `int`.

- That cast truncates rather than rounds. A median of 1.5 becomes 1, not 2.
- Any procedure that can produce a fractional value in an integer column must say which way it goes, and the implementation has to follow that sentence literally.
- Where this note and the spec appear to disagree, **the spec wins**.

5 Common mistakes

1. **Computing statistics before checking types.** Totals come out absurd, or an error appears several steps later. The cause is text that looks like numbers. Audit first and coerce early.
2. **Imputing before deduplicating.** Your numbers disagree with everyone else's, because the duplicates biased the statistic you filled with.
3. **Deleting flagged outliers immediately.** Totals and counts shrink quietly. Report and interpret them; delete only when you can state the reason.
4. **Comparing group means without their counts.** A category with a handful of observations takes first place.
5. **Hard-coding an answer found by inspection.** Fixing a value like "there are 3 outliers" passes on the shipped data and fails on the hidden tests used for grading, which run on data built to give a different answer. Always compute from the input.

6 How this maps to the lab tasks

Table 3. Tasks matched to concept sections. Which section each task draws on, and what grading looks at. Worth checking once before you write the report.

Lab task	Concept	What grading looks at
Task 1. missingness summary	§3.1	audit before repair, per-column counting
Task 2. cleaning procedure	§3.1–3.2	stated order, defensible imputation, derivation over estimation
Task 3. IQR outliers	§3.3	quartile-based fences, flagging versus deleting
Task 4. group statistics	§3.4	split-apply-combine, count next to mean, sort direction
Report figures 1–3	§3.5	each figure answers a stated question

7 Self-check

1. Why must the imputation statistic be computed after removing duplicates? Build a small example dataset to demonstrate why.
2. Give two realistic reasons a numeric column ends up stored as text.
3. The IQR rule flags 40% of a column. What does that tell you about the distribution, and what should you do instead of deleting the flagged values?
4. One category has the highest average rating in your group table. Which single number could make that finding meaningless?
5. For each of the three report figures, write one sentence saying what question it answers.

These questions are fair game for the oral check. You must be able to explain every line you submit.

8 Further reading

- pandas User Guide, Working with missing data
  `https://pandas.pydata.org/docs/user_guide/missing_data.html`
- pandas User Guide, Group by: split-apply-combine
  `https://pandas.pydata.org/docs/user_guide/groupby.html`
- Hadley Wickham, "Tidy Data", *Journal of Statistical Software* 59(10), 2014
  `https://www.jstatsoft.org/article/view/v059i10`